# Measuring LLM Security Verdicts: Coverage, Abstention, and Adversarial Suppression

S.V.Divya
Department of CSE,
K.P.R College of Engineering and Technology,
Coimbatore-642109.
divyasvvsb@gmail.com

R Harshan
Department of CSE,
K.P.R College of Engineering and Technology,
Coimbatore-642109.
harshanr695@gmail.com

P.M Akhil Jeikar
Department of CSE,
K.P.R College of Engineering and Technology,
Coimbatore-642109.
akhiljeikar06@gmail.com

T Ezhilperungo
Department of CSE,
K.P.R College of Engineering and Technology,
Coimbatore-642109.
ezhilperungotamizhazhagan@gmail.com

M Gurunathan
Department of CSE,
K.P.R College of Engineering and Technology,
Coimbatore-642109.
gurunathanmofficial@gmail.com

Abstract—Large language models (LLMs) are increasingly used as verifiers of security findings: a static analyzer proposes a candidate and a model adjudicates whether it is real. Whether such a verdict can be trusted is rarely measured. We adapt the four-way decomposition—an instrument previously used to show that static analyzers flag constructs rather than vulnerabilities—to LLM verifiers and extend it with evidence gating, an adversarial comment attack, a composition study, and a calibration analysis. On a 151-pair real-fix security stratum from JavaVFC, an evidence-grounded verifier tracks the fix on 17.6% of pairs (95% CI [16.7,18.5], $k$=5) against 0.7% for the pattern matcher it is meant to replace and 0% for CodeQL, with exact McNemar $p<10^{-7}$ against both—yet it abstains on 51.5% of judgements and misses 79.1% of genuine fixes, so its bottleneck is coverage, not accuracy. Gating costs no recall (3:4, $p$=1.0) and removes ungrounded positives; a blinded-message ablation shows the commit message leaks −18.1 points of detection, an LLM still detects ∼19× the tool on code alone. Adversarial comments evade 12–20%, every successful evasion is an abstention rather than a false positive, and stripping comments does not help: an attacker need only make the verifier unsure, not wrong. Calibration at scale is mediocre (ECE 0.116) and accuracy on a labelled benchmark does not improve over the analyzer. Our scaled evidence spans one model family on the stratum plus a cross-family replication on the legacy subset, and we state the remaining limits plainly. We release a reproducible harness and argue that abstention and coverage should be reported as first-class outcomes of security agents.

Index Terms—abstention, adversarial robustness, calibration, syntax graphs, evidence grounding, large language models, program analysis, selective prediction, static analysis, vulnerability verification.

## I. Introduction

Automated vulnerability analysis has converged on a standard shape: a policy queries tools and a code graph and is trained or prompted to report findings [1]–[4]. LLM agents now drive that loop—reasoning over repositories, invoking analyzers, and drafting fixes. In each case a model makes verification judgements: given a candidate produced by a scanner, a fuzzer, or another model, is it real? As agents take over the gate between a warning and a fix, the question becomes whether their verdicts can be trusted.

That question is harder than it looks. A model can be right about the code and wrong about the vulnerability: it can flag a dangerous-looking construct that the patch did not touch, agree that a benchmark sample is vulnerable without reasoning about the vulnerability, or become confident on inputs unlike its training distribution. For a static analyzer, our companion study showed that the fraction of commits whose verdict moves with a fix can be two orders of magnitude below its benchmark true-positive rate, and introduced the four-way decomposition to make that gap visible [5]. We ask the same of LLM verifiers.

The stakes are concrete. Static analysis has always lived with false positives [6], and machine-learning vulnerability detectors inherit the same label and validity problems [7]–[9]. When a model is placed in front of remediation, two failures are possible: it can confirm a non-vulnerability (wasted effort, eroded trust) or, more dangerously, suppress a real one. Prior work shows that LLM security detectors are easily steered by adversarial comments [10] and by indirect prompt injection [11]; what has not been measured systematically is whether an LLM verdict tracks the vulnerability at all, how it fails when it does not, and how it behaves under composition and an adversary.
Contributions.

- An instrument. We give the first systematic application of the four-way decomposition — introduced in our companion study for static analyzers [5] — to LLM verifiers, gate every verdict on machine-checkable evidence, split adversarial evasion by outcome (false positive vs. abstention), and report the coverage of the process. The harness, corpora, and pinned analyzers are released.
- A coverage-bound finding. An evidence-grounded verifier tracks the fix 17.6% of the time (95% CI [16.7,18.5], $k$=5) against 0.7% for the pattern matcher (which over-reports 10×) and 0% for CodeQL—exact McNemar $p<10^{-7}$ against both—yet it abstains on 51.5% of judgements and misses 79.1% of genuine fixes. Its bottleneck is silence, not error—a profile that precision-only evaluation cannot see.
- Abstention as the attack surface. Under adversarial comments, every successful evasion is an abstention rather than a false positive; stripping comments as a mitigation does not reduce evasion. An attacker need only make the verifier unsure, not wrong.
- Leakage is measurable. Blanking the commit message drops detection on the same pairs from 31.4% to 13.3% (−18.1 points): the message is a real label channel, and code alone still detects ∼19× the tool.
- Composition relocates trust. Evidence gating costs no recall (3:4, $p$=1.0) and marginally removes ungrounded positives; requiring agreement with a weaker deterministic verifier loses recall and gains nothing.
- A negative result on labelled ground truth. On OWASP Benchmark (2,740 labelled cases) the analyzer reaches recall 0.905 / precision 0.734, but the LLM verifier does not improve precision: in a matched sample neither a local 7B nor a cloud model rejects false positives (0–2 of 15), and blinding the scanner hint only raises abstention. Verification is coverage-bound.
- Calibration and cost. At $n$=151 the verifier is nearly precise (0.9% fixed-side false positives) but not reliably calibrated (ECE 0.116 [0.106, 0.131]); disabling model thinking cuts cost ∼8× at a measured precision cost, and a content-addressed verdict cache removes 100% of repeat cost.

We are explicit about what the scaled evidence does and does not cover (Section VIII) and treat the paper as a measurement-methodology contribution rather than a claim of a large effect.

## II. Background and Related Work

### A. LLM agents for vulnerability analysis

A fast-moving line of systems couples LLMs with static analysis, symbolic execution, and fuzzing. OpenAnt decomposes code and adversarially verifies candidates [4], and agentic pipelines such as VulAgentRL, Antares, and EAVA ground discovery in analyzers and code graphs [1]–[3]. These systems report improved discovery, but they generally do not measure whether a verdict tracks the vulnerability, and they rarely report the coverage–precision trade-off of the verifier itself. We measure the verdict rather than the discovery rate.

### B. Verifiers and verifiable rewards

VulAgentRL uses a code property graph both as a tool and as a reward verifier with exact node matching [1]; Antares trains small models with verifiable rewards [2]; EAVA couples supervised tuning with reinforcement learning and requires evidence for a verdict [3]. "Verifiable" here means that the training signal can be checked, not that the model's inference-time verdict is grounded. We do not train; we ask what trust a grounded verdict earns at inference time and how it fails.

### C. Adversarial robustness of LLM code analysis

ALIBI shows that adversarial source comments defeat LLM vulnerability detectors with high success rates [10]; indirect prompt injection compromises tool-integrated agents [11]; and code security-hardening work demonstrates both the risk and partial defenses [12]. Earlier work established that AI code assistants frequently produce insecure code [13], [14]. We attack the verifier rather than the detector, split evasion by outcome (false positive vs. abstention), and include an adaptive attacker.

### D. Abstention, selective prediction, and calibration

Selective classification makes rejection an explicit learning problem [15], and out-of-distribution detection is the closely related recognition task [16]. Modern neural networks are often miscalibrated [17], and abstention has become a first-class topic for LLMs, where models can partially "know what they know" [18]; recent surveys catalogue the design space [19]. We bring this lens to security verification and show that abstention is not merely a safety valve—it is the dominant adversarial target and the verifier's main bottleneck.

### E. LLM-as-a-judge reliability

Using a model to adjudicate another model's output is now standard [20], [21], and its failure modes are documented: evaluators are order- and format-sensitive [22] and favour their own generations [23]. Security verification is a judge problem in this sense, but the ground truth is a code change rather than a preference; we exploit that difference by scoring verdicts against the fix.

### F. Static analysis false positives and measurement validity

Static analysis has been deployed at scale [6], and its false positives are a persistent obstacle in practice. Machine learning for security inherits methodological pitfalls—leakage, temporal bias, and ill-defined labels [9]—and vulnerability datasets in particular are noisy [8]. Classical and learned vulnerability detectors and their limits are well studied [7], [24], [25]. The OWASP Benchmark is

the standard scanner yardstick [26]; our companion study showed its score is not well defined and does not predict real detection, and introduced the four-way decomposition [5]. We extend [5] from static tools to LLM verifiers.

### G. Code property graphs and query engines

Program-analysis tooling exposes relational views of code: code property graphs [27] and object-oriented query languages over relational data [28] are the basis of modern taint engines. Our evidence gating uses a lightweight tree-sitter graph rather than a sound dataflow engine; Section VIII treats the resulting weakness explicitly.

### H. Ensembles and composition

Combining predictors is a classical idea [29], and diversity is the property that makes an ensemble work [30]. Our composition result is a negative instance of that principle in a security setting: adding a weaker verifier by consensus does not add diversity, it adds the weaker verifier's blind spot.

## III. Method

Figure 1 shows the pipeline end to end. A revision pair is scanned by a pinned analyzer; each candidate becomes a bounded verdict request; the verifier must ground its verdict in resolvable evidence; the vulnerable and fixed verdicts are then decomposed and summarised. An adversary can inject comments before verification.

### A. The four-way decomposition

For a vulnerable/fixed revision pair, we classify a verifier's verdict into one of four cells (Fig. 2). Only vuln-only is evidence of detection; both means the verifier flagged a construct present before and after the fix, so its verdict could not have been about the vulnerability. From the cells we derive the flip rate (vuln-only fraction), the naive coverage (fraction flagged on the vulnerable revision), and their ratio, the over-report factor,

$$\text{over-report} = \frac{\text{naive coverage}}{\text{flip rate}} = \frac{\text{both} + \text{vuln-only}}{\text{vuln-only}}, \quad (1)$$

so "over-reports 6×" means six findings on the vulnerable side for every one that tracks the fix.

### B. Verifier arms

We compare four arms on identical inputs. V1 (LLM, ungated): the model's verdict is taken as-is. V1g (LLM, gated): every cited evidence reference must resolve to a real syntax node, or a positive verdict is downgraded to unknown. V2 (tool): the pinned analyzer's verdict. V5 (hybrid): flagged only if the gated LLM and the tool agree.

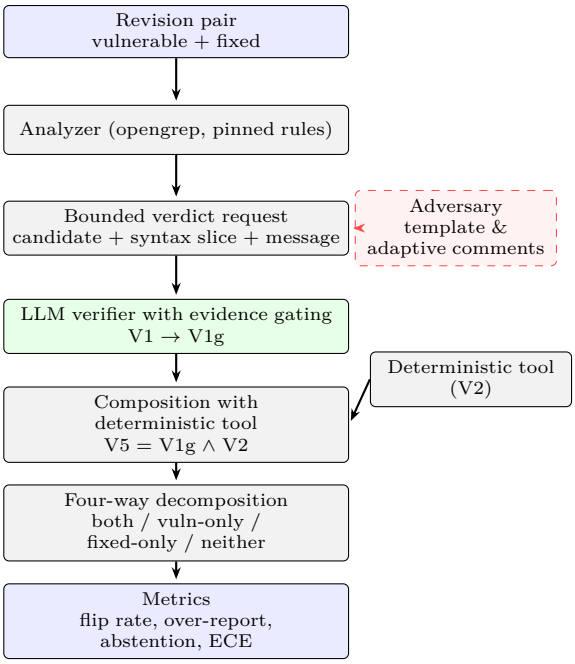



Fig. 1. RewardAudit architecture. A revision pair is scanned by a pinned analyzer; each candidate is turned into a bounded verdict request over a tree-sitter syntax graph and the commit message. The verifier must ground its verdict in resolvable evidence (V1→V1g). An adversary (dashed) can inject comments before verification, and a deterministic tool can be composed with the verifier. Verdicts on the two revisions feed the four-way decomposition, which yields the reported metrics.

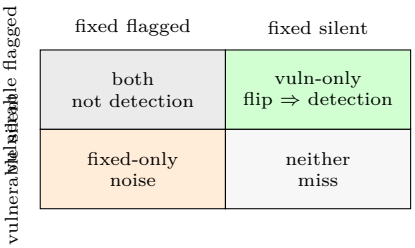



Fig. 2. The four-way decomposition over a revision pair. Only vuln-only is detection; both means the verifier flagged a construct present before and after the fix.

### C. Bounded verdict request and evidence gating

The model never sees a repository. For a candidate it receives a bounded request: the candidate (rule, location, CWE hint), the enclosing function extracted from a tree-sitter syntax graph, and the commit message as legitimate context. It must respond with a strict JSON verdict, cite path:line references, and may answer unknown_risk. Each reference is resolved against the graph; an unverifiable positive is downgraded. Gating checks exactly one thing: that the cited path:line exists and the enclosing function resolves. It does not verify taint flow, reachability, or any semantic property of the data — a lightweight syntax graph is not a dataflow engine, and we do not pretend otherwise (Section VIII). We deliberately never

feed whole files by default, since that is the strawman this measurement is meant to beat.

### D. Adversary

Templates: five plausible comment strategies (authority, sanitizer, tool-output, test-passed, reviewed) inserted just above the suspected sink. Adaptive: an attacker model writes a comment, observes the verifier's verdict, and refines over up to three rounds [10]. We split evasion into false-positive and abstention outcomes, because they target different failure modes and demand different defenses.

### E. Calibration, selective risk, and cost

Per revision judgement we record confidence, correctness, and disposition. We compute expected calibration error (ECE), reliability bins, and a selective-risk/coverage curve [15], plus false-negative rate on vulnerable revisions and false-positive rate on fixed revisions. Cost is reported in tokens and, at known prices, dollars per true positive.

### F. A worked example

Consider an analyzer warning on os.system(command) in a helper reached from an HTTP handler that reads an environment variable. An ungated model may answer true positive with a plausible but unfalsifiable rationale. Under V1g it must cite a graph node for the source and the sink; if the cited nodes do not exist or the call path is not present, the positive is downgraded to unknown_risk. This is the mechanism that produces the coverage–precision profile in Section V: grounding removes unverifiable positives, but a model that cannot produce verifiable evidence abstains, and abstention is scored as a miss on a fixed-side test.

## IV. Experimental Setup

Corpora. (i) JavaVFC [31] (784 commits; 327 usable single-file; 324 materialised offline) screened to a security stratum of 151 revision pairs: a security keyword in the commit subject plus a second out-of-band match (screen version recorded in the manifest), built as a running-100 census plus a 51-pair extension. Every stratum result below uses these 151 pairs unless noted. The hand-picked strong-security subset of 21 pairs from our companion study [5] is retained for cross-family replication and for comparison with the original $k$=5 measurement. (ii) 5 hand-built Python vulnerable/fixed pairs and 3 canaries. (iii) The prior study's 21-file Java control set (18 vulnerable, 3 safe) [5]. Each revision is presented as an isolated file; the commit message is supplied as candidate context, as a bug report would be, and is removed only in the blinded-message ablation.

Analyzers. opengrep 1.30.0 with the prior study's pinned 690-rule Java ruleset (sha256 77df32e0…) and a minimal 5-rule Python ruleset. CodeQL 2.27.0 with the java-security-extended suite runs per pair (both revisions in an isolated no-build database); a whole-directory database is inapplicable on this corpus—the javac extractor is OOM-killed at ∼300 files—and we validated the per-pair adapter by reproducing the prior study's finding on a known pair.

TABLE I
Cross-family verdicts on 5 Python pairs / 3 canaries.

| Model | flip rate | both | canary FP |
|---|---|---|---|
| gpt-oss-120b | 80% | 1 | 1/3 |
| gpt-oss-20b | 100% | 0 | 1/3 |
| qwen3.8-27b | 100% | 0 | 0/3 |
| deepseek-flash | 100% | 0 | 0/3 |
| deepseek-v4-pro | 80% | 0 | 0/3 (1 abstain) |

Models. Five families via an OpenAI-compatible API: gpt-oss-120b, gpt-oss-20b, qwen3.8-27b, deepseek-flash, deepseek-v4-pro. The 151-pair stratum runs use mimo-v2.6-flash ($k$=5, verdict cache disabled); the legacy 21-pair subset is run with both mimo-v2.6-flash and deepseek-flash ($k$=5 each) for the cross-family replication; the remaining families are checked on the 5-pair Python set.

Metrics and statistics. For each arm we report the mean over $k$=5 independent runs with 95% bootstrap intervals; because reasoning models are non-deterministic even at temperature 0, we report the observed range alongside the interval. Per-pair majority verdicts are compared with the exact McNemar test on the same 151 pairs. Over-report is (both + vuln_only)/vuln_only.

Reproducibility. All assets are pinned by hash (pins.json); the control reproduction and the Python-pair measurement run fully offline; 158 unit and integration tests accompany the harness.

## V. Results

### A. Verifier dependence

On identical inputs, models disagree in both flip rate and failure mode; the cross-language check is summarised in Table I. gpt-oss-120b flipped on 4/5 pairs and flagged a safe canary; gpt-oss-20b and qwen and deepseek-flash flipped on 5/5 with one family showing a canary false positive; deepseek-v4-pro abstained on one vulnerable revision. The result is verifier-dependent—the phenomenon the instrument is designed to expose. The same holds at scale: within one configuration repeated five times on the stratum, the flip rate's 95% interval is [16.7, 18.5]% while the legacy 21-pair subset spans 28.6–38.1% across runs (Section VIII), so even one model does not have a stable point value.

### B. LLM versus pattern matcher and CodeQL, and composition

The deterministic baselines barely track the fix at all. On the 151-pair stratum the pattern matcher flips on 0.7% of pairs (1/151) while flagging both revisions on 9 more, so it over-reports 10× its flip rate; CodeQL flips on none (0/151). The evidence-gated LLM tracks the fix on 17.6% [16.7,18.5] over five runs—25× the tool, where CodeQL is

TABLE II
Verifier arms on the 151-pair security stratum (mimo-v2.6-flash, mean over $k$=5, 95% CI; deterministic arms are a single run). Over-report = (both + vuln_only)/vuln_only.

| Arm | flip rate | coverage | over-report |
|---|---|---|---|
| V3 CodeQL | 0.0% | 0.0% | — |
| V2 tool | 0.7% | 6.6% | 10.0× |
| V1 ungated | 18.3% [16.8, 19.5] | 18.8% | 1.03× |
| V1g gated | 17.6% [16.7, 18.5] | 18.0% | 1.02× |

TABLE III
Exact McNemar on majority verdicts, same 151 pairs. Discordant counts are pair-majorities where only one arm flips.

| Comparison | only A / only B | agree | $p$ |
|---|---|---|---|
| V1g vs. V2 tool | 24 / 0 | 127 | $6 \times 10^{-8}$ |
| V1g vs. V3 CodeQL | 25 / 0 | 126 | $6 \times 10^{-8}$ |
| V1 vs. V2 tool | 25 / 0 | 126 | $6 \times 10^{-8}$ |
| V1 vs. V3 CodeQL | 26 / 0 | 125 | $3 \times 10^{-8}$ |
| V1g vs. V1 ungated | 3 / 4 | 144 | 1.0 |
| V2 tool vs. V3 CodeQL | 1 / 0 | 150 | 1.0 |

zero—and, unlike the tool, essentially never over-reports (1.02×). Table II summarises the arms; Table III gives the exact paired tests: both baselines are defeated at $p<10^{-7}$, and the ungated arm matches them too, so the finding is about the LLM, not the gate.

Gating is an honest null: 3:4, $p$=1.0—it costs no recall and marginally reduces ungrounded positives (0.8→0.6 per run). The gate's value is not accuracy but that every accepted verdict answers for itself. The composition ablation points the same way: on the legacy subset, requiring consensus with the tool (V5) loses a flip without removing any both—the weaker verifier's blind spot becomes the composition's—while grounding a claim is free.

CodeQL as a baseline. CodeQL is the reviewers' requested taint-analysis arm. A whole-directory no-build database is invalid on this corpus (the javac extractor is OOM-killed; retries yield a valid empty SARIF—a silent clean), so we run the prior study's per-pair method, whose adapter we validated by reproducing its known finding (java/sensitive-log, CWE-532, on efaa120ecc0d). Across all 151 pairs CodeQL reports zero findings in the vulnerable file of any pair (and 22 with findings only in the fixed file), against the prior study's independent 0/32—under no-build isolated-file conditions, deterministic taint analysis contributes no signal on this corpus that the LLM verifier can be gated against.

## C. Replication and label leakage

Two ablations separate sample, family, and leakage effects on the legacy 21-pair subset (Table IV). First, the same model on the same 21 hand-picked pairs flips at 31.4% (range 28.6–38.1), against 17.6% on the 151-pair census: the drop is a sample effect, not a model change. Second, deepseek-flash on those same 21 pairs

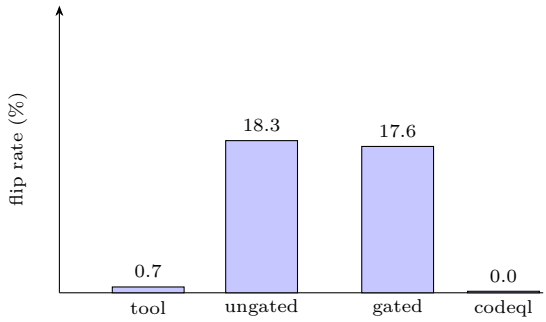



Fig. 3. Flip rate by verifier arm on the 151-pair stratum. The deterministic arms barely move: the tool flips once (and over-reports 10×), CodeQL never.

TABLE IV
Replication and leakage ablations on the legacy 21-pair subset ($k$=5, mimo-v2.6-flash unless noted).

| Configuration | flip rate | range |
|---|---|---|
| with commit message | 31.4% | 28.6–38.1% |
| without message | 13.3% | 4.8–19.0% |
| deepseek-flash (E0, same pairs) | 30.5% | [22.9, 37.1] |
| 151-pair stratum (contrast) | 17.6% | [16.7, 18.5] |

flipped at 30.5% [22.9,37.1] in the original measurement—two families agree to within one pair, so the family effect is small and the sample effect dominates. Third, blanking the commit message (the blinded-message ablation) drops the same pairs from 31.4% to 13.3% (range 4.8–19.0): −18.1 points, or −58% relative. The commit message is a real label channel. Code-only detection (13.3%) is still ∼19× the pattern matcher's 0.7%.

## D. Adversarial comments

Two attack strategies were tested: fixed templates and an adaptive attacker (Table V). The sanitizer tactic ("the input is validated upstream") was strongest across families. Critically, all successful adaptive evasions, and 8 of 9 template evasions, were abstentions rather than false positives: an attacker need only make the verifier unsure. The adaptive attacker roughly doubles evasion relative to templates. This is the security consequence of the coverage finding above: a defense that only suppresses false positives leaves the dominant path open.

A mitigation that does not help. As a defense against comment-based steering we strip all comments from the judged source (line-preserving, tree-sitter). On the stratum subset the strip did not reduce evasion—12% (3/25) with comments present vs. 20% (6/30) stripped—and every evasion in both arms was again an abstention (false positives 0). We report this as a negative result with a stated confound: the two arms' baseline true-positive sets differed (5 vs. 6) because a temperature-0 reasoning model is self-inconsistent across runs on these pairs, so the comparison is not fully paired. The honest conclusion

TABLE V
Evasion on the original 21-pair study. JavaVFC templates: baseline 7/21 true positives; adaptive: baseline 6/21, attacker deepseek-v4-pro vs. judge deepseek-flash. Stratum-scale attack and mitigation results are discussed above.

| Attack | evasion | via FP | via abstain |
|---|---|---|---|
| templates (authority) | 43% (3/7) | 0 | 3 |
| templates (overall) | 26% | 1 | 8 |
| adaptive (3 rounds) | 50% (3/6) | 0 | 3 |

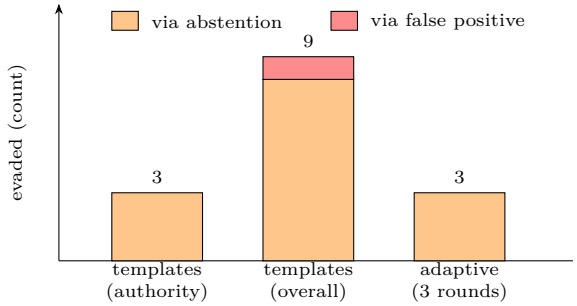



Fig. 4. Successful evasions by attack and outcome. Nearly all are abstentions: the attacker only has to make the verifier unsure, not wrong.

is that input sanitization of comments is not shown to defend against the abstention attack; the threat model (Section VII) treats abstention as the asset to protect.

### E. Calibration and coverage

The verifier is nearly precise but poorly calibrated: over five repeated runs on the 151-pair stratum (Table VI), it false-positives on under 1% of fixed revisions, yet its ECE is 0.116 [0.106, 0.131]—better than the worst values of the original 21-pair measurement (up to 0.35) but still far from a calibrated probability. It abstains on 51.5% of the judgements and misses 79.1% of genuine fixes. The verifier is therefore trustworthy when it commits—but the coverage is low enough that most real fixes are not certified by it at all.

### F. Cost

Output tokens dominate (∼91% of cost). Disabling model thinking reduced output from 121,596 to 5,271 tokens over 42 judgements (∼8× lower cost) but introduced fixed-side false positives (0%→9.5%) and worsened ECE (0.153→0.207). A content-addressed verdict cache keyed on the model, prompts, and slice removed 100% of repeat cost. At off-peak prices the gated verifier costs ≈$0.0017 per judgement. The economics matter: if the bottleneck is coverage, the productive investment is better context and better evidence, not more sampling of the same prompt.

### G. Detection effectiveness on a labelled benchmark

To test the whole pipeline against ground truth we ran it on the OWASP Benchmark for Java [26]: 2,740 labelled test cases (1,415 vulnerable, 1,325 safe). The

TABLE VI
Calibration on the 151-pair stratum, mimo-v2.6-flash, $k$=5 mean [95% CI] (six shards merged).

| Metric | Mean [95% CI] |
|---|---|
| abstention rate | 51.5 [48.5, 54.4] |
| detection (recall on vuln revision) | 20.9 [19.1, 22.8] |
| false-negative rate | 79.1 [77.2, 80.9] |
| fixed-side false-positive rate | 0.9 [0.3, 1.6] |
| accuracy (scored) | 73.9 [72.1, 76.4] |
| ECE | 0.116 [0.106, 0.131] |

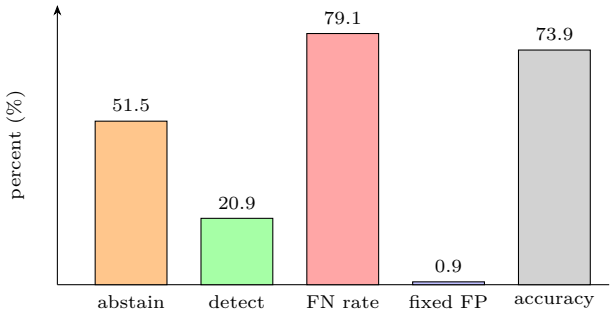



Fig. 5. Calibration and coverage on the stratum (mean over five runs). The verifier is nearly precise (0.9% fixed-side false positives) yet abstains on half the inputs and misses most fixes; ECE is 0.116 [0.106, 0.131].

pinned analyzer alone is a strong detector but over-reports (Table VII).

We then asked whether the LLM verifier improves on that precision. On a matched sample (15 analyzer true positives, 15 analyzer false positives) we ran the verifier with and without the scanner's rule, message, and CWE hint ("blind"), to separate genuine judgement from echoing the scanner (Table VIII).

Two things follow. First, no configuration rejects false positives: neither a local 7B nor the cloud model separates the safe near-misses from the vulnerable cases, and the analyzer's precision is not improved. Second, blinding does not help—removing the scanner's hint does not raise rejection, it drives abstention to 50–73% and cuts recall. The failure is therefore not scanner-confirmation bias but the model's inability to ground the deciding evidence (a sanitizer) from the bounded slice. Error gating aside, verification here is coverage-bound and adds nothing over the analyzer.

## VI. Discussion

The bottleneck is coverage. Across the scaled run the verifier false-positives on under 1% of fixed revisions and abstains on half of all judgements, but its confidence is not reliably calibrated (ECE 0.116) and it misses 79.1% of fixes. Optimising a security agent for precision alone would miss this entirely. We argue abstention should be reported as a first-class outcome alongside precision and recall, and

TABLE VII
Analyzer on 2,740 labelled OWASP Benchmark cases.

| Metric | Value |
|---|---|
| TP / FP / FN / TN | 1,280 / 463 / 135 / 862 |
| recall (TPR) | 0.905 |
| precision | 0.734 |
| false-positive rate | 0.349 |

TABLE VIII
LLM verification on the matched Benchmark sample. A cache hit or a blind run cannot echo the scanner's CWE hint.

| Verifier | TP kept | FP rejected | abstain | prec. |
|---|---|---|---|---|
| local 7B, hinted | 14/15 | 0/15 | 1/30 | 0.48 |
| local 7B, blind | 7/15 | 0/15 | 15/30 | 0.47 |
| cloud, hinted | 10/15 | 2/15 | 13/30 | 0.67 |
| cloud, blind | 5/15 | 0/15 | 22/30 | 0.62 |

that a verifier which silently declines most inputs is not "accurate" in any deployment sense.

Attacks target abstention. Because the verifier already abstains when evidence is thin, an adversary does not need to convince it a vulnerability is safe—only to make it unsure. This reframes adversarial robustness for verifiers: defenses that only suppress false positives leave the dominant path open.

Composition relocates trust. Grounding a claim in verifiable evidence is cheap and strictly helpful here. Requiring consensus with a weaker verifier is not: it inherits that verifier's blind spots. "Add another verifier" is not a reliability strategy; "make the existing verdict answer for itself" is.

Design implications. Three rules follow. (i) Score the fix, not the sample: a verdict should be evaluated by whether it moves with a code change, which is cheap to collect and hard to game. (ii) Make abstention visible: agents should surface the fraction of findings they declined to decide, because that is where adversarial exploitation concentrates. (iii) Prefer grounding to consensus: require a verdict to answer for itself with resolvable evidence rather than adding another model whose agreement may only import a shared blind spot. A stronger form of grounding would require a machine-checkable certificate for every accepted verdict, but our results suggest the immediate constraint is coverage, not the strength of the evidence a model can already produce.

A practical pitfall. A reasoning-model verifier silently degrades to "always abstain" when the token budget is consumed by reasoning tokens (finish_reason=length, empty content). Without the decomposition this is indistinguishable from a null result. We report it because it cost us a run.

## VII. Threat Model

Assets. The verdicts. A verifier sits on the gate between a warning and a fix, so the assets are (i) the integrity of a verdict — it must track the vulnerability, not the surrounding prose — and (ii) its availability — a verdict that silently abstains is a verdict that does not gate anything.

Adversary. Anyone who can influence what the verifier reads: a contributor who embeds adversarial comments in submitted source (measured here), a commit message that names or denies the vulnerability (label leakage, bounded by the blinded-message ablation), and a scanner whose rule, message, and CWE hints may be echoed rather than judged (bounded by the blind condition). We do not model an adversary who can modify the pinned analyzers, the rulesets, or the model provider; those are trusted, pinned by hash, and out of scope.

Trust boundary. All code, comments, scanner output, and commit messages are untrusted data, never instructions. The only components that turn data into a verdict-the-model cannot forge are the evidence gate (path:line resolution against the syntax graph) and the decomposition itself. The abuse cases this paper measures are therefore: steering a verdict into abstention with comments (Section V), ungrounded positives (the gate), and hint-echo (the blind condition).

## VIII. Threats to Validity

Scale. The stratum is 151 screened security pairs from one corpus; the cross-language check is 5 pairs, so the cross-family comparison is directional. Labels. JavaVFC is a screening corpus: our companion audit [5] found only ∼18% of it is a genuine security fix, so recall is a floor against a cleaner label set. Input shape. Each revision is an isolated file, which is a floor for tools that resolve types and near-neutral for pattern matchers. Evidence graph. Our gating uses a tree-sitter graph, not a sound dataflow engine [27], [28]; it can confirm that a cited path:line exists, not that taint flows. Non-determinism. Reasoning models vary run to run at temperature 0; on the legacy subset we observed flip rates 28.6–38.1% across five repeats of one configuration; the $k$=5 bootstrap intervals are reported alongside the observed range. Families at scale. Only mimo-v2.6-flash was repeated on the 151-pair stratum ($k$=5); the cross-family check uses the legacy 21 pairs (mimo 31.4% vs. deepseek 30.5% on identical pairs), which bounds the family effect but does not replace a multi-family census. Contamination. Public fixes may appear in pretraining data; a temporal or private holdout is not yet used. Candidate context. The verdict request carries the commit message as bug-report-like context; a message that names or denies the vulnerability can leak the label directly to the model. The blinded-message ablation (Section V) re-runs the same pairs with the message removed and measures a −18.1-point drop—the leakage is real and quantified, but residual leakage through

code idioms cannot be ruled out. Benchmark. OWASP Benchmark labels are synthetic and single-method, and the verifier ablation is a 30-case sample; it tests precision behaviour, not recall. Adversary. Our templates are fixed; a single adaptive attacker was used, and its adaptivity is shallow. CodeQL baseline. The per-pair no-build mode is the only mode that completes on this corpus; under build-capable conditions CodeQL might cover more, so the 0% flip rate is a measurement of this configuration, not of CodeQL in general.

## IX. Ethics and Responsible Release

The adversarial comment generator is dual-use. This work is defensive: its purpose is to measure and expose a failure mode so that verifiers can be made reportable and robust. We do not release a turnkey attack service; the generator is a small, reproducible component of a measurement harness, and the paper reports defenses (evidence gating, coverage reporting, abstention accounting) alongside the risk.

## X. Reproducibility and Artifact

The harness, pinned analyzers and rule sets, control set, and corpora are released. Coverage is explicit: Java (a 690-rule ruleset), Python and JavaScript/TypeScript (minimal rule sets) are covered; C is grammar-only; other languages are reported unchecked. A whole-function LLM audit surfaces issues without a ruleset (file context curbs over-flagging), and dependency advisories refresh from the live OSV feed into immutable, dated snapshots. All assets are pinned by hash; the control reproduction (12/18 vulnerable, 1/3 safe) and the Python-pair measurement run fully offline; 158 tests accompany the code. A content-addressed verdict cache makes repeat runs deterministic and free.

## XI. Conclusion and Future Work

LLM security verifiers track fixes far better than pattern matchers and CodeQL and are nearly precise when they commit (0.9% fixed-side false positives), but they are not reliably calibrated and are nonetheless silent on about half of their inputs—a weakness that adversarial comments directly amplify, and that composition with deterministic tools does not fix. The actionable reframing is that the bottleneck of a security verifier is coverage, and that its dominant adversarial target is abstention. Future work must (i) replicate the 151-pair census across at least two more model families, (ii) evaluate path-level candidates on multi-file revisions, and (iii) treat abstention and coverage as first-class metrics for security agents.

## References


[1] Y. Li, T. Zhang, J. Liu et al., "Graph is the verifier: Agentic reinforcement learning for interprocedural vulnerability detection," 2026.

[2] S. Vijay, A. Priyanshu, D. Chapoteau et al., "Antares: Foundation models for agentic vulnerability localization," 2026.

[3] S. Pan, Z. Zheng, J. Zhou et al., "Answer is cheap, show me the evidence! augmenting automated vulnerability assessment with evidence," 2026, iSSTA 2026.

[4] N. Korda and G. Evron, "OpenAnt: LLM-powered vulnerability discovery through code decomposition, adversarial verification, and dynamic testing," 2026.

[5] RewardAudit Project, "A benchmark score that nothing can be concluded from: What the OWASP benchmark measures, and why it is not detection capability," Technical report and artifact (sastaudit); introduces the four-way decomposition, 2026, companion study; source of the instrument adapted here.

[6] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, "Lessons from building static analysis tools at Google," Communications of the ACM, 2018.

[7] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" IEEE Transactions on Software Engineering, 2022.

[8] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023.

[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in Proceedings of the 31st USENIX Security Symposium, 2022.

[10] Z. Wu and C. Nita-Rotaru, "ALIBI: Adaptive agentic attacks on LLM-based vulnerability detectors via adversarial code comments," 2026.

[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023.

[12] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023.

[13] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in 2022 IEEE Symposium on Security and Privacy (SP), 2022.

[14] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023.

[15] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in Advances in Neural Information Processing Systems 30, 2017.

[16] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in International Conference on Learning Representations (ICLR), 2017.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proceedings of the 34th International Conference on Machine Learning (ICML), 2017.

[18] S. Kadavath, T. Conerly, A. Askell et al., "Language models (mostly) know what they know," 2022.

[19] B. Wen, J. Yao, S. Feng, C. Xu, Y. Tsvetkov, B. Howe, and L. L. Wang, "Know your limits: A survey of abstention in large language models," Transactions of the Association for Computational Linguistics, 2025.

[20] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems 36, 2023.

[21] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[22] P. Wang, L. Li, L. Chen, Z. Cai, D. Sun, Y. Wang et al., "Large language models are not fair evaluators," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024.

[23] A. Panickssery, S. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in Advances in Neural Information Processing Systems 37, 2024.
[24] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in Advances in Neural Information Processing Systems 32, 2019.
[25] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025.
[26] OWASP Foundation, "OWASP benchmark project," https://owasp.org/www-project-benchmark/, 2026.
[27] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in 2014 IEEE Symposium on Security and Privacy (SP), 2014.
[28] P. Avgustinov, O. de Moor, M. Peyton Jones, and M. Schäfer, "QL: Object-oriented queries on relational data," in 30th European Conference on Object-Oriented Programming (ECOOP), 2016.
[29] L. Breiman, "Bagging predictors," Machine Learning, 1996.
[30] L. I. Kuncheva and C. J. Whitaker, "Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy," Machine Learning, 2003.
[31] JavaVFC authors, "JavaVFC: A corpus of vulnerability-fixing commits for java," Zenodo record 13731781, 2026, 784 records; screening corpus. A companion audit found ∼18% are genuine security fixes.